## [8] Multilingual Semantic Search with Cohere & Langchain

↳ How to use Langchain to efficiently build semantic search app^ns on top of Cohere's multilingual model.

↳ This article looks at how we can use the Cohere Platform & Langchain to build end-to-end pipelines for multilingual semantic search. We start with a basic semantic search example where we import a list of documents, turn them into text embeddings & return the most similar document to a query. Then we build a more complex app^n that can answer questions about a long-form document, such as an article.

[The summary above was generated by Cohere's new Summarize Beta endpoint, with minor editing.]

### Introduction

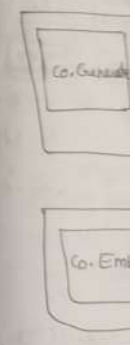
You have an idea for building a cool app^n on top of Cohere's LLMs. But as you start building, you realize there's more to building an LLM-powered app^n than the model alone. You begin building utility utilities around it, but you realize it can take a lot of time, slowing down your productivity. In fact, you have so many ideas in your head. If only there were a way for you to build & test them more efficiently.

In this article, we'll look at how you can achieve that with Cohere & Langchain.

### Overview of Cohere.

The Cohere Platform provides an API for developers & org^s to access cutting-edge LLMs without needing ML know-how. The platform handles all the complexities of curating massive amounts of textual data, model development, distributed training, model serving & more. This means that developers can focus on creating value on the applied side rather than spending time & effort on the capability-building side.

There are 2 key ty
that the Cohere Platfo
and each is served
with text generation,
and get back a sta
prompt. One examp
haiku (the prompt
haiku in return
with text embe
get back a list of
This is useful for
a passage of te
its sentiment.

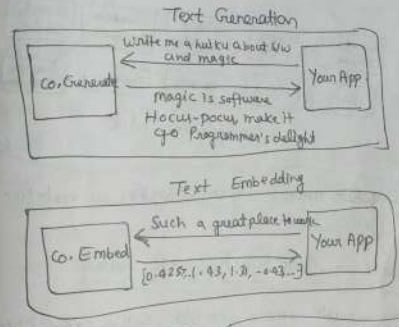



The Two Key
the Cohere

There are 2 key types of language processing capabilities that the Cohere Platform provides – text generation & text embedding and each is served by a diff type of model.

With text generation, we enter a piece of text, an prompt, and get back a stream of text as a completion to the prompt. One example is asking the model to write a haiku (the prompt) & getting an originally written haiku in return (the completion).

With text embedding, we enter a piece of text & get back a list of numbers representing its semantic meaning. This is useful for use cases that involve "measuring" what a passage of text represents, for example, in analyzing its sentiment.

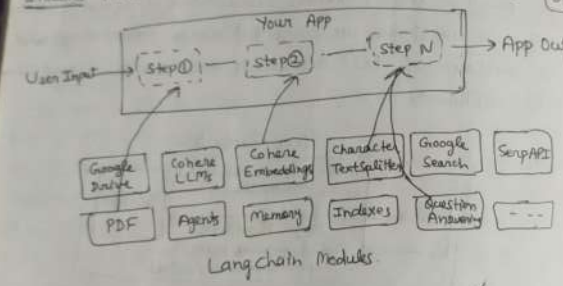



The Two key types of language processing capabilities that the Cohere platform provides

## ↳ Overview of Langchain

To create true value for end users, these models in isolation are often not enough. A truly powerful app emerges when you combine them with other sources of computation or knowledge. This is where Langchain comes into the picture. It is an open-source library that provides abstractions for building LLM-based app^ns. Langchain makes it easy & fast to assemble an end-to-end app^n via its wide range of modules. Think Lego blocks for LLM app^ns.

↳ In this article, we'll explain how to use Langchain's chains and Indexes modules to build end-to-end pipelines with very few lines of code.




Langchain makes it easy to put together an end-to-end LLM-powered app^n.

## ↳ Overview of Multilingual Semantic Search

~~Ques for~~ Semantic search provides search capabilities based on the contextual meaning of a passage of text. It solves the limitation of the alternative method, which is keyword search.

Here's an example. Let's
semantic search system w
"restaurants" because
possible to do that u
will be limited to t

And what is multil

It is a search cap
in the query & the
you would have to b
language, meaning you
inefficient. But wit
model, you need
100+ languages.

When talking
two types of se

→ Multilingual search
are of the same
query of "plan
result of "Be

→ Cross-lingual
result are of
a Hindi query
an English

here's an example. Let's take a query: "places to eat". A semantic search system would be able to associate this with "restaurants" because their meanings are similar. It's not possible to do that with keyword search because the results will be limited to the keywords "places", "to", and "eat".

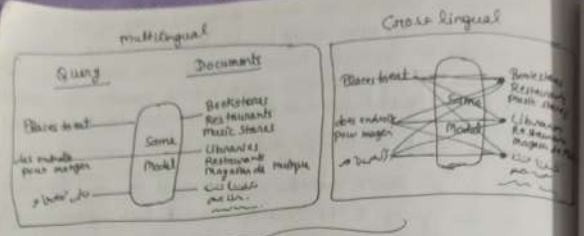
And what is multilingual search?

It is a search capability not limited to one language in the query & the results. To build such systems, traditionally you would have to build individual pipelines for each language, meaning your system will be bloated, costly & inefficient. But with Cohere's multilingual embedding model, you need just one embedding model to handle 100+ languages.

When talking about multilingual models, there are two types of searches: multilingual & cross-lingual.

→ Multilingual search happens when the query & the result are of the same language. For example, an English query of "places to eat" returning an English result of "Bob's Burgers".

→ Cross-lingual search happens when the query & the result are of a different language. For example, a Hindi query of "खाने की जगहें" (places to eat) returning an English result of "Bob's Burgers".




The difference b/w multilingual & cross-lingual searches

With Cohere's multilingual embedding model, we can do both multilingual & cross-lingual searches using one single model

↳ ! pip install cohere langchain qdrant-client tfds-nightly

```
from langchain.embeddings.cohere import CohereEmbeddings
from langchain.llms import Cohere
from langchain.prompts import PromptTemplate
from langchain.text-splitter import RecursiveCharacterTextSplitter
from langchain.chains.question_answering import load_qa_chain
from langchain.chains import RetrievalQA
from langchain.vectorstores import Qdrant
from langchain.document_loaders import TextLoader
import textwrap as tr
import random
import os
os.environ["cohere API Key"] = "Cohere API Key".
```

Example ①:

Docum

1
2
3
4
5
...

Sea

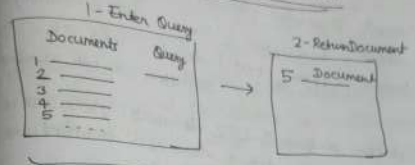
↳ In the
pipeline f
a list
recommend

We'll co
from a
document
While
how g
We'll
the

↳ 'Ste

First, we
from the
classificati

Example ①: Basic Multilingual Search




Searching the most relevant document from a list of documents.

↳ In the 1st example, we'll create a semantic search pipeline for searching the most relevant documents from a list of documents. An example app is a recommendation system for podcasts or articles.

We'll compare a query with a list of documents from a questions dataset, and then return the documents that are most similar to the query. While the documents are in English, we'll see how good the model is at cross-lingual search. We'll enter queries in 5 languages & see if the results match our expectations.

↳ Step ① Import a list of Documents.

First, we import a dataset containing a list of questions from the Text REtrieval Conference (TREC) Question classification dataset. We can take the dataset from

publicly available Tensorflow Datasets library.

```
import tensorflow-datasets as tfds
dataset = tfds.load ('trec', split = 'train')
texts = [item['text'].decode ('utf-8')
                for item in tfds.as_numpy (dataset)]
```

This dataset contains 5,452 documents. Here are a few sample documents from the dataset, where each document is a question:

what is the starting salary for beginning lawyers?
where did Bill Gates go to college?
What task does the Bouvier breed of dog perform?
What are the top boy names in the U.S.?
What is a female rabbit called?

↳ Step② Embed the Documents & Store them in an Index.

Next is where the magic happens. We turn each document into a text embedding, which is a number representation of text that captures its context and meaning. Then we put these embeddings in a vector database (also called "index") so our system can perform the search in a scalable & efficient way.

With Langchain, we can do that with just 2 lines of code. The 1st defines the embeddings model, where we initialize the CohereEmbeddings object with the multilingual model multilingual-22-12.

The second turns th
and indexes them in a
options are available;
offers an open-sou

Here, we define th
save in memory, an
we set the similari
the multilingua
the dot product

# Define the em
embeddings = Co

# Embed the do
db = Qdrant.f

↳ Step③ Enter a

We enter a new search
Now, looking at the
"Bill Gates":

What is Bill
Where did B
How many s

sets library.
tfds
plit = 'train')
utf-8')
in tfds.as_numpy(dataset)

Here are a few sampl
document is a question
beginning lawyers?
f dog perform?
the U.S.?

in an Index.
ch document into
sentation of text
we put these
"Index") so our
& efficient way.

lines of code
initialize the
el model

The second turns the documents into text embeddings & indexes them in a vector database. Several vector databases are available; In our case, we use Qdrant, which offers an open-source vector database.

Here, we define the location of the database, which we save in memory, and the collection name. Also importantly, we set the similarity search method to be 'Dot' as the multilingual embedding model works best with the dot product method.

```
# Define the embeddings model
embeddings = CohereEmbeddings(model="multilingual-22-12")

# Embed the documents & store in index
db = Qdrant.from_texts(texts, embeddings, location=
                       ":memory:", collection=":memory:",
                       collection_name="my_documents",
                       distance_func="Dot")
```

## Step 5 Enter a Query.

We enter a new search query: "How to get in touch with Bill Gates". Now, looking at the documents, there items contain the term "Bill Gates":

What is Bill Gates of Microsoft Email address?
Where did Bill Gates go to college?
How many square feet is Bill Gates' home?

None of these contains exactly the term we are looking for with the same keyword. A traditional keyword search would fail to distinguish the most relevant document b/w theses three.

However, with semantic search, we should expect to get the document "What is Bill Gates of microsoft email address" as the most similar query, because "get in touch" and "email address" imply a similar intent.

But let's not stop there. The multilingual model can also perform well a cross-lingual tasks. So, we'll enter the same query in two other languages: French & Indonesian.

```
queries = ["How to get in touch with Bill Gates",
           "Comment entrer en contact avec
            Bill Gates",
           "Cara menghubungi Bill Gates"]
queries_lang = ["English", "French", "Indonesian"]
```

Let's see how the model performs.

↳ Step 4: Return the Document Most Similar to Query

Now we take the query & pass through the similarity_search method to get the document most similar to the query. Here, we take the page_content attribut of the top-most document, hence the [0] index.

```
answers = [ ]
for query in queries:
    docs = db.similarity_search(query)
    answers.append(docs[0].page_content)
```

And below are the results ~~of the~~ for the query in the three different languages: English, French & Indonesian. The search engine successfully finds the most semantically similar document to the query.

Query language : English

Query : How to get in touch with Bill Gates

Most similar existing question: What is Bill Gates of Microsoft Email address?

- - - -

Query language : French

Query : comment entrer en contact avec Bill Gates

Most similar existing question: What is Bill Gates of Microsoft Email address?

- - - -

Query language : Indonesian

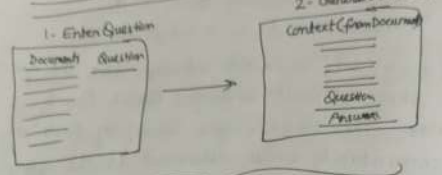
Query : Cara menghubungi Bill Gates

Most similar existing question: What is Bill Gates of Microsoft Email Address?

- - -

In this example, we saw how we can leverage langchain to build a semantic search system with just a few lines of code ; faster than if you were to build it from scratch. This theme continues in following example

## Example ②: Search-Based Question Answering




Answering questions about a long-form document

In this second example, we'll use semantic search to answer questions about a long-form document, such as an article. Here, we take a question from the user, search for the most relevant passages in the document, and use these to generate an answer. An example app is the search bar for a documentation website.

This time, we'll add a text generation model to the pipeline so we can perform the answer generation part.

And to keep it simple, we'll look at an all-English example from the query to the text generation. Having said that, we can use the same multilingual-22-12 model for the embedding part since it is a multilingual model.

### Step①: Add an Article and Chunk It into Smaller Passages

We'll use a single long-form article as an example, which is Steve Jobs' Stanford University 2005 commencement address.